2. Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

Gemini Auto & Claude Sonnet 5

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both GenAI programs through their websites (claude.ai & gemini.google.com respectively)

c. Prompt used. (You must use the same prompt for both GenAIs.)

Modify the Assignment 4 code of the student you're assigned as follows (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

• Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.

• Each solutionN.txt file should include the following:

o If no solutions are found, a single line: "No solution found."

o If solutions are found, it should include all solutions followed by a single line: "Solutions found: n", where n is the number of solutions found.

o The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.

• You must modify the Assignment 4 you receive, not your Assignment 4.

• Do not replace the entire file with your code. Your GTA will use the Github Blame button to check your modifications.

• Add your name to the Profile Comments as "Revised by:" along with a revision date.

• You must get all required functionality from Assignment 4 working, even if the student you inherited it from did not.

HERE IS THE STUDENTS CODE:

d. Generated code from both GenAIs.

GEMINI'S GENERATION:

```
// EECS 348 Assignment 5 - Assignment5.cpp
// Sudoku solver: recursive depth-first search with backtracking (MRV cell ordering).
// Revised by: [Your Name] on October 27, 2026
//
// Build:  g++ -std=c++17 -O2 -Wall -o Assignment5 Assignment5.cpp
// Run:    ./Assignment5                      (solves puzzle1.txt .. puzzle5.txt ->
solution1.txt .. solution5.txt)
//         ./Assignment5 file1.txt file2.txt  (solves explicit files)

#include <array>       // std::array: fixed-size rows of the grid
#include <cctype>      // std::isspace, std::isprint: character classification
```

```cpp
#include <filesystem>  // std::filesystem: check that a file exists and is a regular
file
#include <fstream>     // std::ifstream, std::ofstream: reading puzzle files and
writing solutions
#include <iostream>    // std::cout, std::cerr, std::ostream: error logging
#include <stdexcept>   // std::runtime_error, std::exception: error handling
#include <string>      // std::string: file names and messages
#include <system_error>// std::error_code: non-throwing filesystem checks
#include <vector>      // std::vector: candidate lists and solution lists

// ----------------------------------------------------------------------------
// PuzzleException: the error type thrown when a puzzle file cannot be used.
// ----------------------------------------------------------------------------
class PuzzleException : public std::runtime_error {
public:
   explicit PuzzleException(const std::string &message)
       : std::runtime_error(message) {}
};

// ----------------------------------------------------------------------------
// Board: holds a 9x9 grid and knows how to read, print, and validate itself.
// ----------------------------------------------------------------------------
class Board {
public:
   static const int SIZE = 9;
   static const int BLOCK = 3;

   Board() {
       for (auto &row : grid) row.fill(0);
   }

   void load(const std::string &filename) {
       namespace fs = std::filesystem;
       std::error_code ec;

       if (filename.empty())
           throw PuzzleException("no file name was given");
       if (!fs::exists(filename, ec) || ec)
           throw PuzzleException("file does not exist");
       if (!fs::is_regular_file(filename, ec) || ec)
           throw PuzzleException("path is not a regular file");
```

```
        std::ifstream in(filename, std::ios::binary);
        if (!in)
            throw PuzzleException("file could not be opened (check permissions)");

        char bom[3] = {0, 0, 0};
        in.read(bom, 3);
        bool hasBom = in.gcount() == 3 &&
                      static_cast<unsigned char>(bom[0]) == 0xEF &&
                      static_cast<unsigned char>(bom[1]) == 0xBB &&
                      static_cast<unsigned char>(bom[2]) == 0xBF;
        if (!hasBom) {
            in.clear();
            in.seekg(0);
        }

        std::array<std::array<int, SIZE>, SIZE> temp;
        for (auto &row : temp) row.fill(0);
        int count = 0;
        int line = 1;
        char ch;

        while (in.get(ch)) {
            if (ch == '\n') {
                ++line;
                continue;
            }
            if (std::isspace(static_cast<unsigned char>(ch)))
                continue;

            bool isDigit = (ch >= '1' && ch <= '9');
            bool isBlank = (ch == '_' || ch == '0' || ch == '.');

            if (!isDigit && !isBlank) {
                std::string shown = std::isprint(static_cast<unsigned char>(ch))
                                    ? std::string("'") + ch + "'"
                                    : std::string("a non-printable character");
                throw PuzzleException("invalid character " + shown + " on line " +
std::to_string(line));
            }

            if (count >= SIZE * SIZE)
```

```
                throw PuzzleException("more than 81 cells found (extra data on line " +
std::to_string(line) + ")");

            temp[count / SIZE][count % SIZE] = isDigit ? (ch - '0') : 0;
            ++count;
        }

        if (in.bad())
            throw PuzzleException("I/O error while reading the file");
        if (count == 0)
            throw PuzzleException("file is empty");
        if (count < SIZE * SIZE)
            throw PuzzleException("only " + std::to_string(count) + " cells found;
expected 81");

        grid = temp;
    }

    int get(int r, int c) const { return grid[r][c]; }
    void set(int r, int c, int v) { grid[r][c] = v; }
    bool isEmpty(int r, int c) const { return grid[r][c] == 0; }

    bool canPlace(int r, int c, int v) const {
        for (int i = 0; i < SIZE; ++i) {
            if (grid[r][i] == v) return false;
            if (grid[i][c] == v) return false;
        }
        int br = (r / BLOCK) * BLOCK;
        int bc = (c / BLOCK) * BLOCK;
        for (int i = 0; i < BLOCK; ++i)
            for (int j = 0; j < BLOCK; ++j)
                if (grid[br + i][bc + j] == v) return false;
        return true;
    }

    std::vector<int> candidates(int r, int c) const {
        std::vector<int> result;
        for (int v = 1; v <= SIZE; ++v)
            if (canPlace(r, c, v)) result.push_back(v);
        return result;
    }
```

```
    bool findBestEmptyCell(int &outR, int &outC) const {
        size_t best = SIZE + 1;
        bool found = false;
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (!isEmpty(r, c)) continue;
                size_t n = candidates(r, c).size();
                if (n < best) {
                    best = n;
                    outR = r;
                    outC = c;
                    found = true;
                    if (best == 0) return true;
                }
            }
        }
        return found;
    }

    std::string firstConflict() const {
        Board copy = *this;
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                int v = copy.grid[r][c];
                if (v == 0) continue;
                copy.grid[r][c] = 0;
                bool ok = copy.canPlace(r, c, v);
                copy.grid[r][c] = v;
                if (!ok)
                    return "the given " + std::to_string(v) + " at row " +
std::to_string(r + 1) +
                           ", column " + std::to_string(c + 1) +
                           " conflicts with another " + std::to_string(v) +
                           " in its row, column or block";
            }
        }
        return "";
    }

    // Prints the solution in the same format as the input puzzle files (9 rows of 9
digits)
    void printInputFormat(std::ostream &out) const {
```

```
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                out << grid[r][c];
            }
            out << '\n';
        }
    }

private:
    std::array<std::array<int, SIZE>, SIZE> grid;
};

// ---------------------------------------------------------------------------
// SudokuSolver: recursive DFS with backtracking. Collects solutions.
// ---------------------------------------------------------------------------
class SudokuSolver {
public:
    static const size_t MAX_SOLUTIONS = 100;

    explicit SudokuSolver(const Board &puzzle) : working(puzzle) {}

    const std::vector<Board> &solveAll() {
        solutions.clear();
        limitReached = false;
        conflict = working.firstConflict();
        if (conflict.empty()) search();
        return solutions;
    }

    bool hitLimit() const { return limitReached; }
    const std::string &getConflict() const { return conflict; }

private:
    Board working;
    std::vector<Board> solutions;
    bool limitReached = false;
    std::string conflict;

    void search() {
        if (limitReached) return;

        int r = 0, c = 0;
```

```
        if (!working.findBestEmptyCell(r, c)) {
            solutions.push_back(working);
            if (solutions.size() >= MAX_SOLUTIONS) limitReached = true;
            return;
        }

        std::vector<int> options = working.candidates(r, c);
        for (int v : options) {
            working.set(r, c, v);
            search();
            working.set(r, c, 0);
            if (limitReached) return;
        }
    }
};

// ----------------------------------------------------------------------------
// Application: processes puzzle files and writes solutions to solutionN.txt
// ----------------------------------------------------------------------------
class Application {
public:
    Application(int argc, char *argv[]) {
        if (argc > 1) {
            for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
        } else {
            for (int i = 1; i <= 5; ++i)
                files.push_back("puzzle" + std::to_string(i) + ".txt");
        }
    }

    int run() const {
        int failures = 0;
        for (size_t i = 0; i < files.size(); ++i) {
            std::string filename = files[i];
            int n = static_cast<int>(i + 1);
            if (filename.rfind("puzzle", 0) == 0) {
                try {
                    size_t start = 6;
                    size_t end = filename.find(".txt");
                    if (end != std::string::npos && end > start) {
                        n = std::stoi(filename.substr(start, end - start));
                    }
```

```
            } catch (...) {}
        }

        std::string outFilename = "solution" + std::to_string(n) + ".txt";
        std::ofstream outFile(outFilename);
        if (!outFile) {
            std::cerr << "ERROR: Could not open output file " << outFilename <<
"\n";
            ++failures;
            continue;
        }

        Board puzzle;
        try {
            puzzle.load(filename);
        } catch (const std::exception &) {
            // If loading or parsing fails, output required single line format
            outFile << "No solution found.\n";
            ++failures;
            continue;
        }

        SudokuSolver solver(puzzle);
        const std::vector<Board> &solutions = solver.solveAll();

        if (solutions.empty()) {
            outFile << "No solution found.\n";
        } else {
            for (const auto &sol : solutions) {
                sol.printInputFormat(outFile);
            }
            outFile << "Solutions found: " << solutions.size() << "\n";
        }
    }
    return failures > 0 ? 1 : 0;
  }

private:
  std::vector<std::string> files;
};

int main(int argc, char *argv[]) {
```

```
    try {
        return Application(argc, argv).run();
    } catch (const std::exception &e) {
        std::cerr << "Fatal error: " << e.what() << '\n';
        return 2;
    }
}
```

CLAUDE'S GENERATION:

```
#include <array>
#include <cctype>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
#include <system_error>
#include <vector>

class PuzzleException : public std::runtime_error {
public:
    explicit PuzzleException(const std::string &message)
        : std::runtime_error(message) {}
};

class Board {
public:
    static const int SIZE = 9;
    static const int BLOCK = 3;

    Board() {
        for (auto &row : grid) row.fill(0);
    }

    void load(const std::string &filename) {
        namespace fs = std::filesystem;
        std::error_code ec;

        if (filename.empty())
            throw PuzzleException("no file name was given");
        if (!fs::exists(filename, ec) || ec)
            throw PuzzleException("file does not exist");
```

```
        if (!fs::is_regular_file(filename, ec) || ec)
            throw PuzzleException("path is not a regular file");

        std::ifstream in(filename, std::ios::binary);
        if (!in)
            throw PuzzleException("file could not be opened (check permissions)");

        char bom[3] = {0, 0, 0};
        in.read(bom, 3);
        bool hasBom = in.gcount() == 3 &&
                      static_cast<unsigned char>(bom[0]) == 0xEF &&
                      static_cast<unsigned char>(bom[1]) == 0xBB &&
                      static_cast<unsigned char>(bom[2]) == 0xBF;
        if (!hasBom) {
            in.clear();
            in.seekg(0);
        }

        std::array<std::array<int, SIZE>, SIZE> temp;
        for (auto &row : temp) row.fill(0);
        int count = 0;
        int line = 1;
        char ch;

        while (in.get(ch)) {
            if (ch == '\n') {
                ++line;
                continue;
            }
            if (std::isspace(static_cast<unsigned char>(ch)))
                continue;

            bool isDigit = (ch >= '1' && ch <= '9');
            bool isBlank = (ch == '_' || ch == '0' || ch == '.');

            if (!isDigit && !isBlank) {
                std::string shown = std::isprint(static_cast<unsigned char>(ch))
                                    ? std::string("'") + ch + "'"
                                    : std::string("a non-printable character");
                throw PuzzleException("invalid character " + shown + " on line " +
std::to_string(line));
            }
```

```
            if (count >= SIZE * SIZE)
                throw PuzzleException("more than 81 cells found (extra data on line " +
std::to_string(line) + ")");

            temp[count / SIZE][count % SIZE] = isDigit ? (ch - '0') : 0;
            ++count;
        }

        if (in.bad())
            throw PuzzleException("I/O error while reading the file");
        if (count == 0)
            throw PuzzleException("file is empty");
        if (count < SIZE * SIZE)
            throw PuzzleException("only " + std::to_string(count) + " cells found;
expected 81");

        grid = temp;
    }

    int get(int r, int c) const { return grid[r][c]; }
    void set(int r, int c, int v) { grid[r][c] = v; }
    bool isEmpty(int r, int c) const { return grid[r][c] == 0; }

    bool canPlace(int r, int c, int v) const {
        for (int i = 0; i < SIZE; ++i) {
            if (grid[r][i] == v) return false;
            if (grid[i][c] == v) return false;
        }
        int br = (r / BLOCK) * BLOCK;
        int bc = (c / BLOCK) * BLOCK;
        for (int i = 0; i < BLOCK; ++i)
            for (int j = 0; j < BLOCK; ++j)
                if (grid[br + i][bc + j] == v) return false;
        return true;
    }

    std::vector<int> candidates(int r, int c) const {
        std::vector<int> result;
        for (int v = 1; v <= SIZE; ++v)
            if (canPlace(r, c, v)) result.push_back(v);
        return result;
```

```
    }

    bool findBestEmptyCell(int &outR, int &outC) const {
        size_t best = SIZE + 1;
        bool found = false;
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (!isEmpty(r, c)) continue;
                size_t n = candidates(r, c).size();
                if (n < best) {
                    best = n;
                    outR = r;
                    outC = c;
                    found = true;
                    if (best == 0) return true;
                }
            }
        }
        return found;
    }

    std::string firstConflict() const {
        Board copy = *this;
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                int v = copy.grid[r][c];
                if (v == 0) continue;
                copy.grid[r][c] = 0;
                bool ok = copy.canPlace(r, c, v);
                copy.grid[r][c] = v;
                if (!ok)
                    return "the given " + std::to_string(v) + " at row " +
std::to_string(r + 1) +
                           ", column " + std::to_string(c + 1) +
                           " conflicts with another " + std::to_string(v) +
                           " in its row, column or block";
            }
        }
        return "";
    }

    void print(std::ostream &out) const {
```

```
        for (int r = 0; r < SIZE; ++r) {
            if (r > 0 && r % BLOCK == 0) out << "------+-------+------\n";
            for (int c = 0; c < SIZE; ++c) {
                if (c > 0 && c % BLOCK == 0) out << "| ";
                out << (grid[r][c] == 0 ? '_' : static_cast<char>('0' + grid[r][c]));
                if (c < SIZE - 1) out << ' ';
            }
            out << '\n';
        }
    }

    void printPlain(std::ostream &out) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                out << (grid[r][c] == 0 ? '_' : static_cast<char>('0' + grid[r][c]));
                if (c < SIZE - 1) out << ' ';
            }
            out << '\n';
        }
    }

private:
    std::array<std::array<int, SIZE>, SIZE> grid;
};

class SudokuSolver {
public:

    static const size_t MAX_SOLUTIONS = 100;

    explicit SudokuSolver(const Board &puzzle) : working(puzzle) {}

    const std::vector<Board> &solveAll() {
        solutions.clear();
        limitReached = false;
        conflict = working.firstConflict();
        if (conflict.empty()) search();
        return solutions;
    }

    bool hitLimit() const { return limitReached; }
    const std::string &getConflict() const { return conflict; }
```

```
private:
    Board working;
    std::vector<Board> solutions;
    bool limitReached = false;
    std::string conflict;

    void search() {
        if (limitReached) return;

        int r = 0, c = 0;
        if (!working.findBestEmptyCell(r, c)) {
            solutions.push_back(working);
            if (solutions.size() >= MAX_SOLUTIONS) limitReached = true;
            return;
        }

        std::vector<int> options = working.candidates(r, c);
        for (int v : options) {
            working.set(r, c, v);
            search();
            working.set(r, c, 0);
            if (limitReached) return;
        }
    }
};

class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string &file) : filename(file) {}

    bool run(std::ostream &out) const {
        out << "==============================\n";
        out << "Puzzle file: " << filename << "\n\n";

        Board puzzle;
        try {
            puzzle.load(filename);
        } catch (const PuzzleException &e) {
            out << "ERROR: " << e.what() << "\n\n";
            return false;
        } catch (const std::exception &e) {
```

```
            out << "ERROR: unexpected problem while reading: " << e.what() << "\n\n";
            return false;
        }

        out << "Puzzle:\n";
        puzzle.print(out);
        out << '\n';

        try {
            SudokuSolver solver(puzzle);
            const std::vector<Board> &solutions = solver.solveAll();
            writeSolutionFile(solutions, out);

            if (solutions.empty()) {
                out << "No solution found\n";
                if (!solver.getConflict().empty())
                    out << "Reason: " << solver.getConflict() << "\n";
                out << '\n';
                return true;
            }

            out << solutions.size()
                << (solutions.size() == 1 ? " solution found:\n\n" : " solutions
found:\n\n");
            for (size_t i = 0; i < solutions.size(); ++i) {
                if (solutions.size() > 1) out << "Solution " << (i + 1) << ":\n";
                else out << "Solution:\n";
                solutions[i].print(out);
                out << '\n';
            }

            if (solver.hitLimit())
                out << "Note: stopped after " << SudokuSolver::MAX_SOLUTIONS
                    << " solutions (the puzzle may have more).\n\n";
        } catch (const std::exception &e) {
            out << "ERROR: unexpected problem while solving: " << e.what() << "\n\n";
            return false;
        }
        return true;
    }

private:
```

```
    std::string filename;

    std::string solutionFileName() const {
        std::string base = filename;
        size_t slash = base.find_last_of("/\\");
        if (slash != std::string::npos) base = base.substr(slash + 1);
        size_t dot = base.find_last_of('.');
        std::string stem = (dot == std::string::npos) ? base : base.substr(0, dot);
        size_t start = stem.size();
        while (start > 0 && std::isdigit(static_cast<unsigned char>(stem[start - 1])))
            --start;
        std::string number = stem.substr(start);
        if (number.empty()) return "solution_" + stem + ".txt";
        return "solution" + number + ".txt";
    }

    void writeSolutionFile(const std::vector<Board> &solutions, std::ostream &out)
const {
        std::string name = solutionFileName();
        std::ofstream file(name);
        if (!file) {
            out << "ERROR: could not create " << name << "\n";
            return;
        }
        if (solutions.empty()) {
            file << "No solution found.\n";
        } else {
            for (size_t i = 0; i < solutions.size(); ++i) {
                if (i > 0) file << '\n';
                solutions[i].printPlain(file);
            }
            file << "Solutions found: " << solutions.size() << "\n";
        }
        if (!file) out << "ERROR: failed while writing " << name << "\n";
        else out << "Results written to " << name << "\n\n";
    }
};

class Application {
public:
    Application(int argc, char *argv[]) {
        if (argc > 1) {
```

```
            for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
        } else {
            for (int i = 1; i <= 5; ++i)
                files.push_back("puzzle" + std::to_string(i) + ".txt");
        }
    }

    int run() const {
        int failures = 0;
        for (const std::string &f : files)
            if (!PuzzleRunner(f).run(std::cout)) ++failures;
        if (failures > 0)
            std::cout << failures << " file(s) could not be processed.\n";
        return failures > 0 ? 1 : 0;
    }

private:
    std::vector<std::string> files;
};

int main(int argc, char *argv[]) {
    try {
        return Application(argc, argv).run();
    } catch (const std::exception &e) {
        std::cerr << "Fatal error: " << e.what() << '\n';
        return 2;
    }
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness: Both Claude and Gemini use recursive backtracking and the Minimum Remaining Values (MRV) heuristic to solve Sudoku puzzles. Both programs validate the input files and check for conflicts between the given numbers. However, there are differences in how they handle the output. Claude preserves the original program's console report, prints the puzzle and its solutions, and reports errors when a file cannot be opened. Gemini prints nothing to the screen and writes “No solution found.” to the output file when a file is missing, without displaying an error message. Gemini also prints the solution grids without spaces between the numbers, which does not match the input file format. Another issue is that both generated programs limit the number of solutions to 100, meaning they can miss valid solutions when a puzzle has more than 100. Overall, Claude's output and error handling are more consistent with the original program and assignment requirements.

Execution Time: Both programs use recursive backtracking and the MRV heuristic, which helps reduce the number of possibilities that need to be explored. However, both repeatedly calculate possible candidates and create vectors while searching for the next cell to fill. This creates some additional work and memory allocation during the search. Neither program has a speed advantage; both solve the five assignment puzzles in a few milliseconds. In terms of asymptotic time complexity, both programs have a worst-case exponential running time because they use recursive backtracking to explore possible Sudoku solutions. If E represents the number of empty cells in a 9-by-9 puzzle, a loose upper bound for the search is O(9^E), since each empty cell can have up to nine possible values. The MRV heuristic can reduce the number of possibilities explored in practice, but it does not eliminate the exponential worst-case complexity. Since the board has a fixed size, checking possible values and selecting the next cell adds constant-bounded work per search step. Both programs therefore have the same general worst-case time complexity.

Space Complexity: Both programs use a fixed-size 9-by-9 board and recursive calls to explore possible solutions. They also store discovered solutions in a vector, with a maximum of 100 solutions. This means they use additional memory to store the solutions instead of processing each one immediately. The recursion stack also uses memory as the solver explores different possibilities. The 100-solution limit reduces the amount of memory used for stored solutions, but it also prevents the programs from finding every solution for puzzles that have more than 100. In terms of auxiliary space complexity, the recursion stack uses O(E) space, where E is the number of empty cells, and storing S solution boards adds O(S) space because each board has a fixed size. Therefore, the total space is O(E + S), with S capped at 100 by the generated programs.

Maintainability: Claude organizes its program into separate classes for the board, Sudoku solver, puzzle runner, and application. This makes the code easier to understand because each class has a specific responsibility. It also preserves the original program's console output and includes separate methods for handling tasks such as printing the board and generating output filenames. Gemini organizes much of its file-processing and output logic inside the Application class, which makes some responsibilities less separated. More importantly, Gemini removed the student's comments from almost every line of the original program. Only 86 of the original 338 lines remain unchanged, which conflicts with the assignment's requirement to modify the existing file rather than replace most of it. Claude is the better starting point because its organization and preservation of the original program make it easier to revise and maintain.

f. Select one to use as the basis for this assignment and justify why you picked it.
I chose Claude's generated program because it was a better starting point for modifying the original student's code. It preserved the console report, handled file errors more clearly, and wrote the solution grids in the same format as the puzzle files, which

Gemini's version did not. Its separate classes also made the code easier to follow and modify. Both programs had a limit of 100 solutions, so that issue still needed to be fixed. Claude's version was the better choice because it provided a more suitable foundation while allowing me to make the required changes to the existing file instead of rewriting the entire program.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I removed the limit of 100 solutions so the program could continue searching until it found every solution. I kept the required solutionN.txt output format, including writing all solution grids followed by the total number of solutions found, or writing only "No solution found." when there are no solutions. I also preserved the input validation and conflict checks. I tested the revised program, and it produces identical solution files for all five puzzles. solution1.txt matches the provided solution. I also tested a puzzle with 19,824 solutions: the original would have stopped at 100, and the revised version found and wrote all of them.

Execution Time: I changed the candidate-counting process by adding countCandidates(), which uses an integer counter instead of creating a vector of possible values for each cell. The search also tries numbers from 1 through 9 directly and checks whether each number can be placed. These changes reduce unnecessary vector allocations while keeping the MRV heuristic. The revised program still has a worst-case exponential search time, commonly bounded by O(9^E), where E is the number of empty cells in a 9-by-9 puzzle. MRV can reduce the number of possibilities explored in practice, but it does not eliminate the exponential worst-case complexity. My changes reduce overhead rather than changing the algorithm's worst-case asymptotic time complexity. On the five assignment puzzles, both versions finish in a few milliseconds, so the performance benefit is more likely to matter on puzzles that require a larger search.

Space Complexity: I changed the program so that each solution is written to the console and output file as soon as it is found, rather than storing every solution in a vector. I also replaced the candidate vector used for counting possibilities with an integer counter. This reduces the extra memory needed for candidate lists and stored solutions. The program still uses memory for the Sudoku board, the recursive search stack, and the list of input filenames, but it no longer needs to store all the solution boards in memory. These changes reduce the auxiliary space complexity from O(E + S) to O(E), because the program no longer stores all the solution boards in memory. This means the memory used by the solver no longer grows with the number of solutions found, even though the revised program can find more than 100 solutions.

Maintainability: I kept the original class structure and made targeted changes instead of replacing the entire program. I added countCandidates() to count possible values without creating a vector and recordSolution() to handle each solution as it is found, and I kept Claude's printPlain() and solutionFileName() methods for the output format and file names. I also updated the prologue comments to describe the revisions and added

comments to explain the main changes. These revisions make the program easier to follow while preserving the original design and making sure the solver can find and write every solution.